Harshini Dinesh and Samyutha Srinivasan

Dr. Eric Gerber

DS 4420

16 April, 2026

Leveraging a Convolutional Neural Network and Naive Bayesian Classifier for Acute Myeloid Leukemia (AML) Cancer Cell Classification

Introduction

Acute Myeloid Leukemia (AML) is a rapid-onset blood cancer characterized by significant complexity, which makes confirming a diagnosis difficult using traditional clinical and statistical methods. Madduru & Kallipalli [3] address this problem by attempting numerous computational approaches to diagnosing AML, including traditional statistical and tree-based methods like Gradient Boosting Trees, chi-squared testing, information gain, and more. They also identify deep learning (CNNs and RNNs) and ensemble models as potential options to uncover disease patterns, but do not provide any experimental results for these approaches. Our study intends to fill this gap in information by constructing a Python-based CNN model applied to single-cell blood smear images, as well as a Bayesian classifier on clinical metadata, together aiming to improve AML subtype identification.

Literature Review

Regarding image data, Al-Nusair et al. [1] provide a strong argument for using deep learning through their paper. Their review surveyed numerous studies that applied deep learning models, including CNNs, to analyze bone marrow smear images and predict AML patient outcomes. Across the studies they surveyed, CNNs especially excelled at picking up subtle morphological features in cell imagery that traditional methods overlooked. These conclusions are particularly helpful because of the similarity in datasets (since they both involve single-cell imagery) and supports our expectation that implementing a CNN model for this problem statement will lead to strong conclusive results.

In terms of how helpful clinical data is for determining AML subtype and patient risk, Yan et al. [2] conducted a study directly relevant to this area. The study analyzed patient-level data (not images) by testing 101 combinations of ML algorithms, including SVMs and random survival forests, across five independent patient cohorts and yielded strong, consistent predictive accuracy. Although the study did not mention any attempts at using a Bayesian classifier, this draws a similar connection to our study by supporting our use of clinical metadata in the Bayesian classifier to stratify AML subtypes probabilistically.

Overall, the main gaps this study aims to fill are the manual implementation of a CNN to analyze single-cell blood smear images and the use of a Bayesian classifier on patient-level clinical metadata to diagnose AML. Additionally, none of the reviewed studies applied these methods to the AML-Cytomorphology MLL Helmholtz dataset, making this study a novel application of both approaches to a clinically relevant single-cell image collection. Improved automated detection of AML cell types has direct implications for faster and more consistent diagnosis across patient populations and clinical settings.

ML Methodology and Data Collection

Data Collection & Preprocessing

The Convolutional Neural Network model is trained on single-cell images that were collected and presented in the AML-Cytomorphology MLL Helmholtz dataset [4] from The Cancer Imaging Archive. The dataset consists of 81,214 single-cell images collected and digitized by the Munich Leukemia Laboratory (MLL) using 189 peripheral blood smears from 2009 to 2020. These single-cell images consist of one control group of healthy stem cell donors and four prevalent AML subtypes with genetic abnormalities and morphological features: (1) APL with PML::RARA fusion, (2) AML with NPM1 mutation, (3) AML with CBFB::MYH11 fusion (without NPM1 mutation), and (4) AML with RUNX1::RUNX1T1 fusion. To properly collect these single-cell images, the Munich Leukemia Laboratory scanned the blood smears with 10x magnification, creating overview images, and then used the Metasystems Metafer platform for automatic cell detection, and finally scanned 99 to 500 white blood cells per patient in 40x magnification through oil immersion microscopy in .TIF format.

Data collection required downloading the histopathology TIFF images using the IBM Aspera server software. The accompanying .sums file was then parsed in Python to extract the metadata, so images and labels could be loaded via filepaths. Pre-processing included resizing images from 144x144 to 96x96 pixels, converting them to grayscale to reduce computational complexity, and normalizing pixel values to range [0, 1]. Since multiple images come from the same patient, using an image-level split for training and testing for the CNN would have introduced data leakage and falsely improved accuracy and model performance. Thus, images were grouped by patient ID prior to performing a patient-level 75/25 train-test split where the model would be trained on images from 75% of patients and tested on 25% of images from new, unseen patients. For computational stability, the dataset was limited to 15,000 training and 4,000 testing images, and class-weighted loss was used to prevent class imbalance within the data (i.e. weights are set to be inversely proportional to class frequency to represent all AML subtypes equally).

The Bayesian classifier is trained on patient-level clinical metadata included with the AML-Cytomorphology MLL Helmholtz dataset [4]. Each observation represents one of the 189 patients and includes 13 peripheral blood differential features (e.g. age, leucocyte count per µl, and percentage counts for myeloblasts, promyelocytes, myelocytes, neutrophils, monocytes etc.) and a binary categorical feature representing sex (1 = female, 2 = male). The output variable, bag_label, corresponds to one of five AML subtypes: CBFB-MYH11, control, NPM1, PML-RARA, and RUNX1-RUNX1T1. The patient-level clinical metadata is processed with a 80/20 train-test split, and both sets use stratified sampling on bag_label to maintain class proportions. Missing and NaN values in numeric features were imputed by calculating median per feature from the training set, which was then applied to the test set to prevent data leakage.

Convolutional Neural Network (CNN)

Model, Training, & Prediction Methods

The Convolutional Neural Network model was designed to learn the hierarchical spatial features of different single-cell AML images and was constructed with three convolutional layers(32, 64, and 128

filters) using ReLu activation. Each convolutional block used a two-dimensional convolution to ensure that the model could detect local patterns like edges, textures, and the cellular structures, and batch normalization was used after each convolution to stabilize the training. Instead of flattening, Global Average Pooling was implemented to reduce overfitting and a dropout rate of 0.4 was added to improve generalization of the CNN model. Since this was mult-class and not a typical binary classification, a softmax output layer was used to properly identify the 4 AML subtypes and 1 control group. To minimize cross-entropy loss, the model was compiled using an Adam optimizer with a learning rate of 10^{-4}. The model was then fitted to training data for 10 epochs and evaluated on unseen test data to assess performance and generalization capability.

After computing evaluation metrics, the CNN model generated predictions on test-set images, which were visualized by displaying 9 randomly selected images alongside both their true and predicted labels. This qualitative assessment uses inverse label mapping (numeric to class name) to compare model predictions against ground truth and identify correct classifications and misclassifications across AML subtypes.

Naive Bayes

Model, Training, & Prediction Methods

The model is a Gaussian Naive Bayes classifier manually implemented in R for computing the posterior probability of each AML subtype given a patient's blood counts: $P(\text{class} \mid x) \propto P(\text{class}) \times \prod P(x_i \mid \text{class})$. The numeric feature likelihoods were modeled as Gaussians parameterized by means and standard deviations computed for each class, with Laplace-smoothed probabilities for the categorical sex feature. The prediction was assigned to the subtype with the highest log-posterior.

To train this model, the prior probabilities P(class) are estimated as empirical class frequencies, and the likelihood for each feature $P(x_i \mid \text{class})$ is modeled as a Gaussian distribution parameterized by the per-class sample mean and standard deviation. A Gaussian distribution was used to reflect the assumption that each blood count feature follows a bell curve within each subtype–for instance, the model may learn that control patients have a mean myeloblast percentage near zero with a small spread, while PML-RARA patients have a much higher mean with greater variability. The sex feature is handled separately as a categorical variable, with its likelihood estimated as the observed class-conditional proportion from the training data.

At prediction time, the model computes a log-posterior score for each subtype by summing the log-prior and all log-likelihood. The model also assumes that, given the class, all features are conditionally independent, meaning that knowing a patient's myeloblast count provides no additional information about their neutrophil count. While this simplification is not necessarily guaranteed for all cases since blood cell counts are known to be correlated, this substantially reduces the number of parameters to estimate and makes the model tractable on a small dataset of 189 patients.

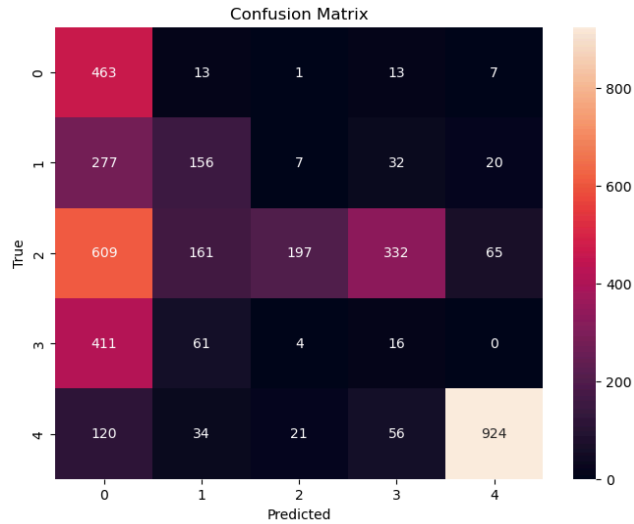
<u>ML Results and Interpretation</u>

Convolutional Neural Network

On the test set of 4,000 images, the Convolutional Neural Network model achieved an overall accuracy of 43.9%. This exceeds the 20% accuracy (⅕ classes) expected from uniform random guessing in a

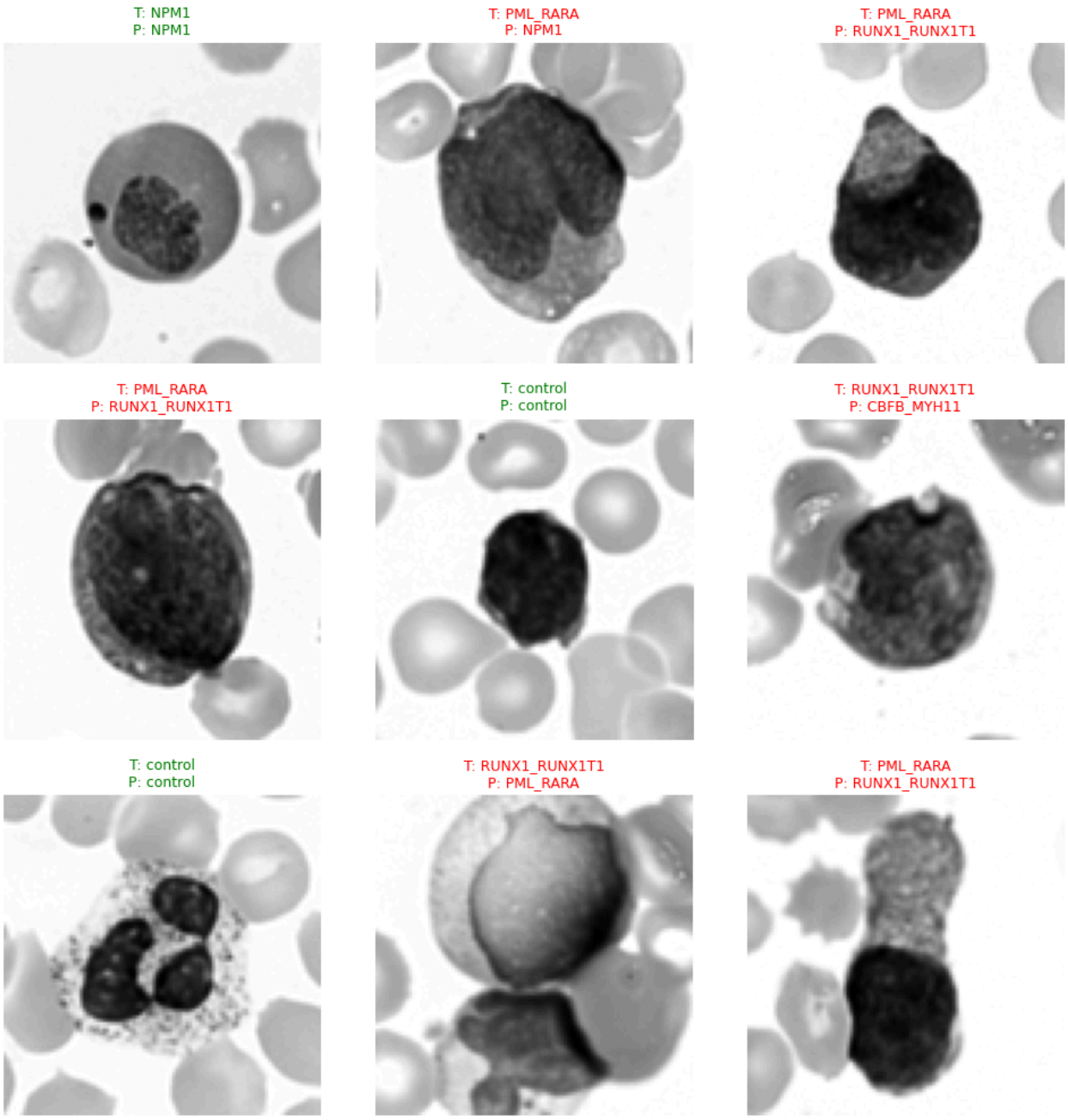
five-class problem, demonstrating moderate predictive performance on a challenging multi-class image classification problem. Training accuracy increased steadily from 32.6% to 53.3% over the 10 epochs and validation accuracy peaked at epoch 7 at 47.08% before fluctuating, suggesting that the model has moderate overfitting where the model improves on training data over time but still struggles to generalize and make accurate predictions on new, unseen samples.

	precision	recall	f1-score	support
0	0.25	0.93	0.39	497
1	0.37	0.32	0.34	492
2	0.86	0.14	0.25	1364
3	0.04	0.03	0.03	492
4	0.91	0.80	0.85	1155
accuracy			0.44	4000
macro avg	0.48	0.45	0.37	4000
weighted avg	0.63	0.44	0.42	4000



Looking at the classification report metrics, the model performed strongly on one subtype (Class 4) with a precision of 0.91 and recall of 0.80, suggesting that this class contains images with unique morphological features that are easier for the CNN to differentiate and identify. However, Class 0 showed a high recall of 0.93 but low precision of 0.25, suggesting that the model over-predicts this class often. The confusion matrix further emphasized these modeling issues and highlighted how there is systematic misclassification between morphologically similar AML subtypes. Specifically, Class 4 shows strong performance with 924 correct predictions out of 1155 samples, meanwhile Class 0 absorbs a significant amount of misclassified observations with 277 samples from Class 1, 609 from Class 2, and 411 from Class 3 being misclassified as Class 0. Thus, the model is biased towards predicting a small subset of the five classes (i.e. Class 0) and needs more than just the features it learned to accurately differentiate classes.

The CNN has moderate predictive performance due to several factors including morphological overlap between AML subtypes causing difficult visual differentiation, information loss from grayscale conversion and downsampling, limited training with only 10 epochs, residual class imbalance even with class weights, and absence of clinical metadata with strong predictive signals. Essentially, the CNN demonstrates the ability to learn morphological features from single-cell AML images but struggles to fully differentiate AML subtypes using image data alone. The visualization of 9 test-set images with both true and predicted labels highlights this behavior, showing a mix of correct classifications and misclassifications across subtypes:

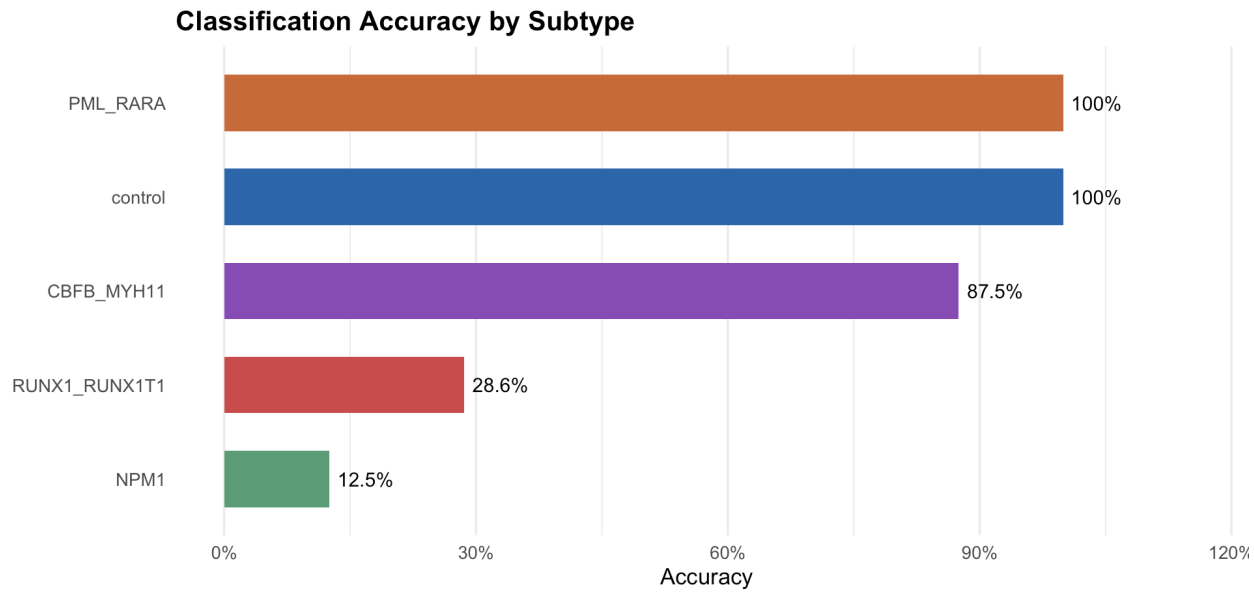


While the model correctly identifies certain classes (e.g., control and NPM1), it frequently confuses morphologically similar subtypes such as PML-RARA and RUNX1-RUNX1T1. These patterns are consistent with the confusion matrix and classification report, reinforcing that class overlap limits performance in five-class classification.

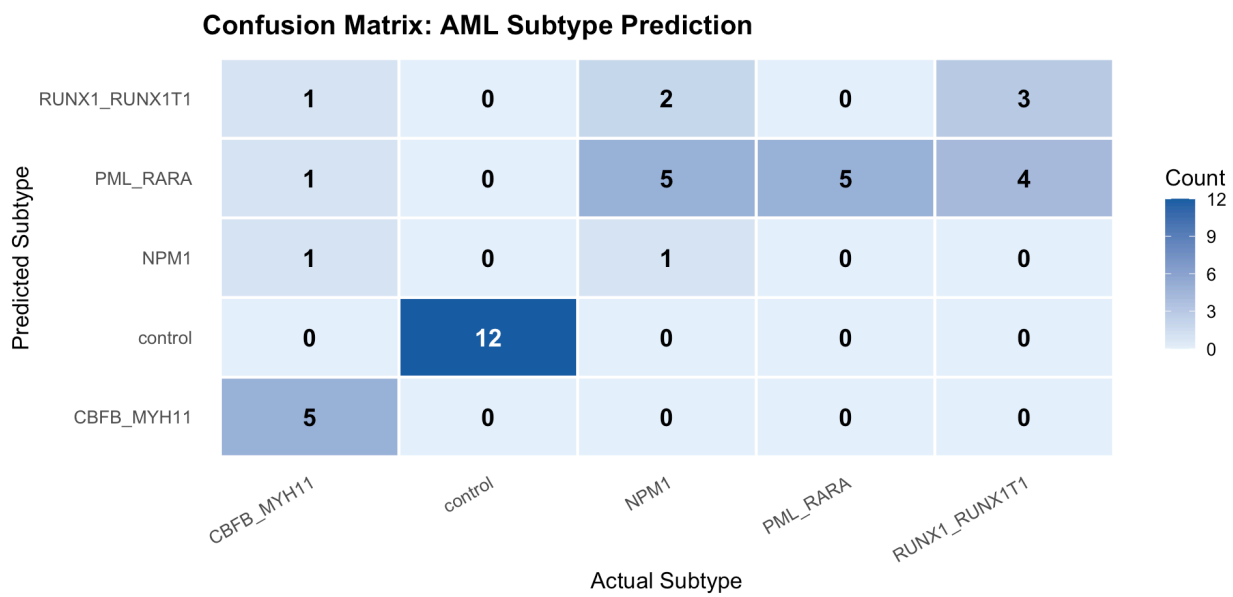
Naive Bayes

On the 20% test set, the Gaussian Naive Bayes classifier achieved an overall accuracy of 65%, with performance varying substantially across different subtypes. The model achieved perfect classification of healthy controls (100%) and PML-RARA (100%), reflecting that these classes are well-separated from

others in the peripheral blood feature space. However, CBFB-MYH11 achieved 62.5% per-class accuracy, RUNX1-RUNX1T1 reached 42.9%, and NPM1 had 12.5%. As the results show, NPM1 was the most difficult subtype to classify, with an observed majority of NPM1 cases being misclassified as PML-RARA.

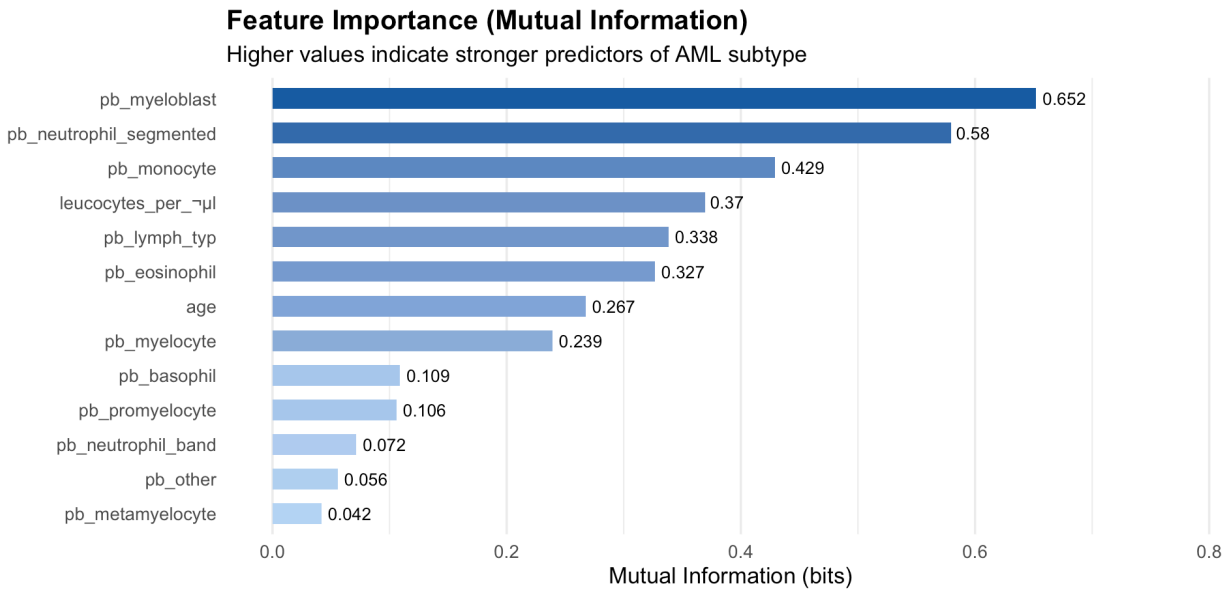


Stratified 10-fold cross-validation was also performed on this dataset to assess the reliability of the accuracies computed for each subtype. The mean CV accuracy was 60.0% (SD = 8.2%), with fold-level accuracy ranging from 40.0% to 68.4%. The confusion matrix also highlighted that PML-RARA was the dominant predicted class, absorbing misclassified cases from both NPM1 (5 out of 8) and RUNX1-RUNX1T1 (4 out of 7). This pattern is consistent across both evaluation strategies, indicating a genuine model limitation rather than a split-specific artifact.



To identify how much of an impact each feature had on classification, mutual information (MI) was computed between each feature and the class label on the training data. For this study, the purpose of

using MI is to measure how much knowing the value of a feature reduces uncertainty about the AML subtype (where zero indicates the feature is uninformative and higher scores indicate stronger subtype-specific signal). From these computations, the three most discriminative features were pb_myeloblast (0.652), pb_neutrophil_segmented (0.58), and pb_monocyte (0.429), which carried the greatest reduction in class uncertainty and are consistent with known clinical patterns. On the other hand, features such as pb_neutrophil_band (0.072), pb_metamyelocyte (0.042), and pb_other (0.056) returned MI scores closer to zero, indicating they had little impact on the results and may introduce noise into the classifier.



Conclusions and Future Work

Convolutional Neural Network

The Convolutional Neural Network model achieved a test accuracy of 43.9% on AML subtype classification, exceeding the 20% accuracy expected from uniform random guessing in a five-class problem. However, both the classification report and confusion matrix demonstrate that morphological features alone are insufficient for high-accuracy classification across all subtypes. The CNN is able to capture spatial structure in cell morphology which would not be possible on a traditional Naive Bayes model, but the results show key limitations when it comes to distinguishing morphologically similar AML subtypes and learning from imbalance data. The confusion matrix shows this with the model's strong prediction bias towards Class 0 and struggle with accurate predictions. It also needs to be noted that the CNN model was based on the assumption that splitting data on the patient-level would prevent dependence between samples, but since multiple cells were taken from the same patients, there could be residual correlation which tampers with the model's ability to make accurate subtype predictions.

Future work should highly consider using a CNN model with richer feature extraction like ResNet and incorporating data augmentation (i.e. rotations, horizontal shifts etc.) to improve generalization. It should be noted that at one point the CNN model presented had attempted data augmentation, but it disrupted performance and decreased accuracy so severely that it was removed. Perhaps, data augmentation will perform better on ResNet which has a pretrained CNN architecture that incorporates the transfer learning machine learning technique for feature extraction and better generalization. Alternatively, the CNN and Naive Bayes classifier models could be combined into a multimodal framework that integrates images for

cell shape and clinical data for blood measurement to improve performance and more accurately classify different AML subtypes.

Naive Bayes

The Gaussian Naive Bayes classifier achieved 65% accuracy on a held-out test set and 60.0% mean accuracy under 10-fold cross-validation for AML subtype classification from peripheral blood differential counts alone. The model reliably separated healthy controls and PML-RARA from other subtypes, reflecting that these classes have distinct peripheral blood signatures. However, it struggled to distinguish NPM1 and RUNX1-RUNX1T1, which overlap substantially in individual feature values, demonstrating how these features are more differentiable through their combinations of features than through any single measurement on its own. Because of this limitation, future work should explore models that capture feature covariance–one example is Linear Discriminant Analysis, which estimates a shared covariance matrix across features. Validating on larger datasets would also help assess whether the 65% accuracy generalizes beyond this 189-patient dataset. Combining the patient-level Bayesian classifier with CNN's cell-level morphological predictions into a unified probabilistic framework also represents a promising direction, as the two models operate on complementary sources of information.

Multimodal model (hybrid approach of CNN and Naive Bayes):

CNN output: $P\ (y|\ image)$

Naive Bayes output: $P\ (y|\ clinical)$

Multimodal Framework: $P\ (y|\ x_{image},\ x_{clinical})\ \alpha\ P\ (y|\ x_{image})\ \cdot\ P\ (y|\ x_{clinical})$

Appendix A: Convolutional Neural Network (CNN) Implementation

```python
# builds a CNN model with deeper convolutional layers, max pooling, regularization, and fully connected layers for classification
inpx = Input(shape=(img_rows, img_cols, 1))

# initial conv layer to detect low-level features (edges, textures)
conv_layer = Conv2D(32, (3,3), activation='relu', padding='same')(inpx)
# stabilizes training and improves convergence
conv_layer = BatchNormalization()(conv_layer)
# reduced from (3,3) to (2,2) to keep more spatial info
pool_layer = MaxPooling2D((2,2))(conv_layer)

# added a second convolutional layer to increase the model's capacity
# and learn more complex features from the images, which can improve classification performance
second_conv_layer = Conv2D(64, (3,3), activation='relu', padding='same')(pool_layer)
# added batch normalization to help with accuracy
second_conv_layer = BatchNormalization()(second_conv_layer)
second_pool_layer = MaxPooling2D((2,2))(second_conv_layer)

#  added 3rd deeper convolutional layer to identify higher-level patterns in AML cells which morph
third_conv_layer = Conv2D(128, (3,3), activation='relu', padding='same')(second_pool_layer)
third_conv_layer = BatchNormalization()(third_conv_layer)
third_pool_layer = MaxPooling2D((2,2))(third_conv_layer)

# flattens to convert the 2D feature maps into a 1D vector
# uses GlobalAveragePooling2D() to reduce overfitting
flat_G = GlobalAveragePooling2D()(third_pool_layer)

# added first hidden layer with 128 neurons
# and 'relu' activation for model complexity
hid_layer = Dense(128, activation='relu')(flat_G)
# used Dropout to prevent overfitting of the medical data
hid_layer = Dropout(0.4)(hid_layer)

# added a second hidden layer with 64 neurons
# changed from 'tanh' activation to 'relu' for better gradient
hid_layer2 = Dense(64, activation='relu')(hid_layer)

# output layer with softmax activation for multi-class classification, where the number of neurons corresponds to the number of unique labels
# we do this because AML has multiple subtypes, so we need to classify into more than two categories (unlike binary), and softmax allows us to get probabilities for each class
out_layer = Dense(len(unique_labels), activation='softmax')(hid_layer2)

model = Model([inpx], out_layer)

# compiles the model using the Adam optimizer for less cross-entropy loss and tracks accuracy as a metric
# added a learning rate for more stable training
model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```python
# fits the model to training data for 10 epochs
# model will learn from data and adjust weights to minimize loss, with batch size of 128 for faster/efficient training
# validation used to help check performance on new, unseen data
model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test, y_test),
    class_weight=class_weights,
    verbose=1
)
```

Appendix B: Naive Bayes Implementation

```r
# Training
# Compute prior probabilities
priors <- table(train_data$bag_label) / nrow(train_data)
subtypes <- names(priors)

# Compute likelihood params
model_params <- list()

for(type in subtypes) {
  subset_df <- train_data[train_data$bag_label == type, ]

  # Estimate Gaussian parameters (Mean and SD) for numeric features
  params <- lapply(subset_df[, numeric_features], function(x) {
    list(mean = mean(x), sd = sd(x) + 0.0001) # prevent divide by zero issues
  })

  # Estimate categorical distribution for 'sex'
  params$sex <- table(factor(subset_df$sex_1f_2m, levels = c(1, 2))) / nrow(subset_df)

  model_params[[type]] <- params
}

# Prediction
predict_aml_bayesian <- function(patient_row, subtypes, priors, params) {
  log_posteriors <- numeric(length(subtypes))
  names(log_posteriors) <- subtypes

  for(type in subtypes) {
    log_p <- log(priors[type])

    # Add log-likelihoods of numeric features
    for(feat in numeric_features) {
      val <- as.numeric(patient_row[feat])
      mean_val <- params[[type]][[feat]]$mean
      sd_val   <- params[[type]][[feat]]$sd
      log_p <- log_p + dnorm(val, mean = mean_val, sd = sd_val, log = TRUE)
    }

    # Add log-likelihood of categorical data
    sex_val <- as.character(patient_row["sex_1f_2m"])
    prob_sex <- params[[type]]$sex[sex_val]
    log_p <- log_p + log(max(prob_sex, 0.001))

    log_posteriors[type] <- log_p
  }

  # Convert back to probabilities
  probs <- exp(log_posteriors - max(log_posteriors))
  probs <- probs / sum(probs)

  return(probs)
}

# Evaluation
# Generate predictions on test set
test_predictions <- t(apply(test_data, 1, function(row) {
  predict_aml_bayesian(row, subtypes, priors, model_params)
}))
```

References

[1] J. Al-Nusair, L. Lanino, A. Durmaz, M. G. Della Porta, A. M. Zeidan, and T. Kewan, "Artificial intelligence in myeloid malignancies: Clinical applications of machine learning in myelodysplastic syndromes and acute myeloid leukemia," *Blood Reviews*, vol. 74, 2025, Art. no. 101340. doi: 10.1016/j.blre.2025.101340.

[2] L. Yan, H. Yu, X. Xu, and M. Liu, "Integrated machine learning-based establishment of a prognostic model in multicenter cohorts for acute myeloid leukemia," *Frontiers in Oncology*, vol. 15, Art. no. 1649594, 2025, doi: 10.3389/fonc.2025.1649594.

[3] Soumya Madduru, "Impact of Machine Learning in Acute Myeloid Leukemia (AML) with Prognosis Approach for Better Accuracy", Int J Intell Syst Appl Eng, vol. 12, no. 3, pp. 3290–3295, May 2024.

[4] Hehr, M., Sadafi, A., Matek, C., Lienemann, P., Pohlkamp, C., Haferlach, T., Spiekermann, K., & Marr, C. (2023). A morphological dataset of white blood cells from patients with four different genetic AML entities and non-malignant controls (AML-Cytomorphology_MLL_Helmholtz) (Version 1) [Data set]. The Cancer Imaging Archive. https://doi.org/10.7937/6PPE-4020